

信息安全作业

202400120045 刘哲旭 通信二班

作业要求：

- 1、一共八个题，每题实际分数 10 分，即 10/80。
- 2、凡涉及到程序的，可以采用 matlab 及 C 语言编写，必须通过编译，有正确的运行结果；运行结果错误，扣 2 分，编译不通过，扣 4 分，如有雷同，扣 6 分。
- 3、请将运行正确的程序代码，拷贝到作业文档中，最终作业文档以 pdf 格式提交。
- 4、非程序作业，请直接在作业文档上完成，给出合理步骤以及结果，最终作业文档以 pdf 格式提交。
- 5、作业的提交包括两个，程序和作业文档（八个题归总为一个文档，包括程序代码和非程序作业），请放置在一个文件夹里，文件夹和作业文档的命名均为：姓名+学号+班级+信息安全.pdf。

-----可以采用 matlab, C 等语言完成)

1、密码分析（单表代换）：

(1) 密文 1:

UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZVUEPHZHMDZSHZQWSFPAPPDTSVPQUZW
YMXUZUHSXEPYEPOPPPSZUFPOMBZWPFUPZHMDJUDTMOHMQ

通过搜索可得以下命令：

```
ciphertext=['UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZVUEPHZHMDZSHZQWSFPAP  
PDTSPVQUZWYMXUZUHSXEPYEPOPPPSZUFPOMBZWPFUPZHMDJUDTMOHMQ'];  
tabulate(ciphertext)  
  
str =  
'UZQSOVUOHXMOPVGPOZPEVSGZWSZOPFPESXUDBMETSXAIZVUEPHZHMDZSHZQWSFPAPPDTSVPQUZWYM  
XUZUHSXEPYEPOPPPSZUFPOMBZWPFUPZHMDJUDTMOHMQ';  
n = 2;  
double_letters = cellstr(reshape(str, [], n));  
freq_table = tabulate(double_letters);  
disp(freq_table);
```

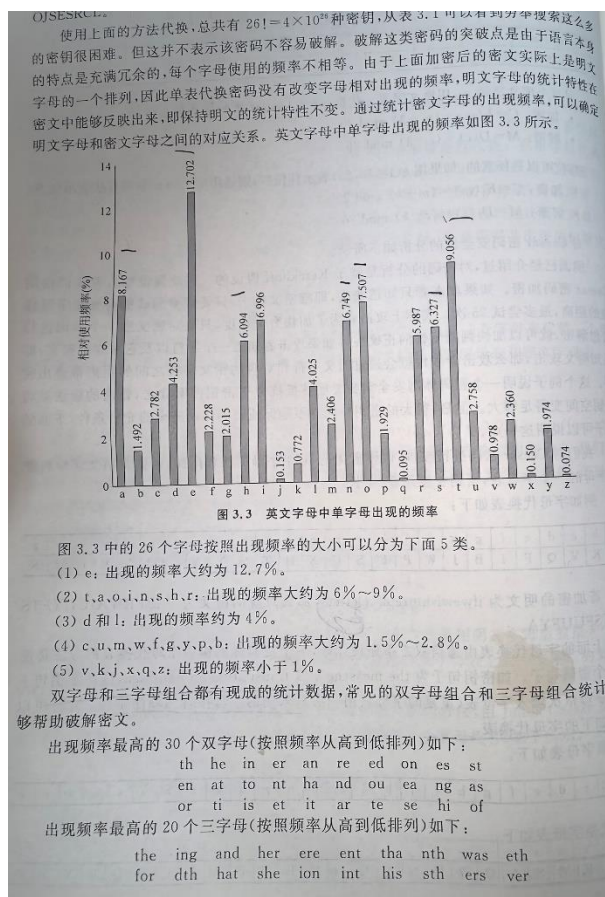
按以上代码统计字母得：

Value	Count	Percent
-------	-------	---------

U	10	8.33%
Z	14	11.67%
Q	4	3.33%
S	10	8.33%
O	8	6.67%
V	5	4.17%
H	7	5.83%
X	5	4.17%
M	8	6.67%
P	17	14.17%
G	2	1.67%
E	6	5.00%
W	4	3.33%
F	4	3.33%
D	5	4.17%
B	2	1.67%
T	3	2.50%
A	2	1.67%
I	1	0.83%
Y	2	1.67%
J	1	0.83%
'US'	[1]	[1.6667]
'ZF'	[2]	[3.3333]
'QP'	[1]	[1.6667]
'SA'	[1]	[1.6667]
'OP'	[1]	[1.6667]
'VP'	[1]	[1.6667]
'UD'	[1]	[1.6667]
'OT'	[1]	[1.6667]
'HS'	[1]	[1.6667]
'XV'	[1]	[1.6667]
'MP'	[2]	[3.3333]
'OQ'	[1]	[1.6667]
'PU'	[2]	[3.3333]
'VZ'	[1]	[1.6667]
'GW'	[1]	[1.6667]
'PY'	[1]	[1.6667]
'OM'	[1]	[1.6667]
'ZX'	[2]	[3.3333]
'EZ'	[2]	[3.3333]
'VU'	[2]	[3.3333]
'SH'	[1]	[1.6667]
'GS'	[1]	[1.6667]
'WE'	[1]	[1.6667]

' SP'	[1]	[1. 6667]
' ZY'	[1]	[1. 6667]
' OE'	[1]	[1. 6667]
' PP'	[2]	[3. 3333]
' FO'	[1]	[1. 6667]
' EP'	[1]	[1. 6667]
' SZ'	[1]	[1. 6667]
' XS'	[1]	[1. 6667]
' UZ'	[1]	[1. 6667]
' DU'	[1]	[1. 6667]
' BF'	[1]	[1. 6667]
' EO'	[1]	[1. 6667]
' TM'	[1]	[1. 6667]
' SB'	[1]	[1. 6667]
' XZ'	[1]	[1. 6667]
' AW'	[1]	[1. 6667]
' IP'	[1]	[1. 6667]
' UP'	[1]	[1. 6667]
' PH'	[1]	[1. 6667]
' HM'	[1]	[1. 6667]
' ZD'	[1]	[1. 6667]
' HJ'	[1]	[1. 6667]
' MU'	[1]	[1. 6667]
' DD'	[1]	[1. 6667]
' ZT'	[1]	[1. 6667]
' SM'	[1]	[1. 6667]
' HO'	[1]	[1. 6667]
' ZH'	[1]	[1. 6667]
' QM'	[1]	[1. 6667]
' WQ'	[1]	[1. 6667]

由单字符可得，p 和 z 出现次数较高，而 uz、sx 在双字母内出现频率较高。



考虑到出现最高频率的单字符是 e 和 t, 而两字母组合最常见为 th 和 he, 不妨假设 p 为 e, 而 z 为 t。替换后得到:

UtQSOVUOHXMOeVGeOteEVSGtWStOeFeESXUDBMETSXAItVUEeHtHMDtSHtQWSFeAeeDTSVeQuTW
YMXUtUHSXEEYeOeetStUFeOMBtWeFUetHMDJUDTMOHMQ

首字母 U 后面跟 t, 可以猜测 U 为 n 或 i, n 作为首字母并且是 nt 开头不常见, 故先假设代表 i, 替换后得到:

itQSOViOHXMOeVGeOteEVSGtWStOeFeESXiDBMETSXAItViEeHtHMDtSHtQWSFeAeeDTSVeQitW
YMXitiHSXEEYeOeetStiFeOMBtWeFietHMDJidTMOHMQ

注意到 e 附近经常有 o 出现, 根据两字母组合里常见匹配可得 se 和 es 出现概率都高, 可以猜测 o 为 s, 替换后得到:

itQSSVisHXMseVGesteEVSGtWStseFeESXiDBMETSXAItViEeHtHMDtSHtQWSFeAeeDTSVeQitW
YMXitiHSXEEYeeseetStiFesMBtWeFietHMDJidTMsHMq

注意到 GesteEVSG 形似 yesterday。故进行替换得

itQSSdisHXMsedyesterdaySyTWStseFerSXidBMrTSXAItDireHtHMDtSHtQWSFeAeeDTSdeQitW
YMXitiHSXreYreseetStiFesMBtWeFietHMDJidTMsHMq

itQSS 形似 it was, 进行替换得:

itwasdisHXMsedyesterdaytWtseFeraXiDBMrTaXAItDireHtHMDtaHtwWaFeAeeDTadewitW
YMXitiHaXreYreseetatiFesMBtWeFietHMDJidTMsHMw

tWt 形似 that, 故进行替换得:

itwasdisHXMsedyesterdaythatseFeraXiDBMrTaXAItDireHtHMDtaHtwhaFeAeeDTadewith
YMXitiHaXreYreseetatiFesMBtheFietHMDJidTMsHMw

direHt 形似 direct, 故进行替换得:

itwasdiscXMsedyesterdaythatseFeraXiDBMrTaXAItdirectcMDtactwhaFeAeeDTadewith
YMXiticaXreYreseetatiFesMBtheFietcMDJiDTMscMw

seFeraX 形似 several, 替换得:

itwasdisclMsedyesterdaythatseFeraliDBMrTalAItdirectcMDtactwhaFeAeeDTadewith
YMliticalreYreseetatiFesMBtheFietcMDJiDTMscMw

reYreseetative 形似 representative, 替换得:

itwasdisclMsedyesterdaythatseveralIDBMrTalAItdirectcMDtactwhaveAeeDTadewith
pMliticalrepreseetativesMBthevietcMDJiDTMscMw

haveAeeD 形似 have been 或者 have seen, 将 D 替换成 n 得:

itwasdisclMsedyesterdaythatseveralinBMrTalAItdirectcMntactwhaveAeenTadewith
pMliticalrepreseetativesMBthevietcMnJinTMscMw

注意到 pMlitical 形似 political, cMntact 形似 contact 替换得:

itwasViscloseVGesterVaGthatseveralIDBorTalAItVirectcontactwhaveAeenTaVewith
politicalrepreseetativesoBthevietcoDJiToscow

结尾地名推测为 Moscow, 替换得:

itwasdisclosedyesterdaythatseveralinBormalAItdirectcontactwhaveAeenmadewith
politicalrepreseetativesoBthevietconJinmoscow

inBormal 形似 informal, 替换得:

itwasdisclosedyesterdaythatseveralinformalAItdirectcontactwhaveAeenmadewith
politicalrepreseetativesofthevietconJinmoscow

两个形容词之间用 and/but, AI 以及 have Aeen, 将 A 替换成 b, I 替换成 u

itwasdisclosedyesterdaythatseveralinformalbutdirectcontactwhavebeenmadewith
politicalrepreseetativesofthevietconJinmoscow

尝试断句得到:

It was disclosed yesterday that several informal but direct contactw have
been made with political represeetatives of the vietconJ in Moscow

最后的 vietconJ in Moscow 通过搜索可以得到如下地名。

国内版 国际版

Bing

Search vietconJ in Moscow

All Images Videos

About 543,000 results

Including results for **vietconG** in Moscow.
Do you want results only for vietconJ in Moscow?

Search Tool
Turn off Hover Ti

+ 7 925 101 9489

Embassy of the Socialist Republic of Vietnam in the Russian Federation
Tel for appointment: **+ 7 925 101 9489** (11h-13h Monday - Thursday)
Hotline for Vietnamese citizen protection: +7 925 101 8368 Fax: +7 499
245 2459 Email: vnconsul.ru@gmail.com Copyright by Vietnam Ministry
of Foreign Affairs

Embassy of the Socialist Republic of Vietnam in the Russian Federation
vnembassy-moscow.mofa.gov.vn/en-us/embassy/Contact/Pages/default.aspx

Was this helpful? 👍 🗑

最后整理得到:

it was disclosed yesterday that several informal but direct contact have been made with political representatives of the Vietcong in Moscow

(2) 密文 2:

```
DLHDRXHLLDLBRXHLLKSMFVRDNHXXVDLRWFHAYCHDLCYAKXHMKGXXHMFQKMRKNDODNRSYIXKCFYI
KMYXZKAMRKNODOKAIVFQKDLDFDHXMFHFKHLABYHXDLNYLFSHMFFYFQYMKDLGQDNFQKSKDMLYDL
RWFAYCHDLMRKNDODKAMWNQRXHLLKSMHSKNHXXKAAYCHDLDLAKRKLAKLFFYKCRQHMDFQKOHNFQFH
FFQKVNHLMYXZKRXHLLDLBRSYIXKCMOSYCHGDAKSHLBKYOAYCHDLMFVRDNHXXKPHCRXKMYOAYCHDL
MHSKIXYNUFMHNUDLBXYBDMFDMGYSUOXYGCHLHBKCKLFHLASYIYFFHMURXHLLDLBQKLNKHMDBX
KAYCHDLDLAKRKLAKLFRXHLLKSNHLIKWMKAFYMYXZKRXHLLDLBRSYIXKCMDLHXXFQKMKZHSDYWMA
YCHDLYLFQKYFQKSQHLAHSYWFKRXHLLKSDMFVRDNHXYOHAYCHDLMRKNDODNRXHLLKS
```

```
ciphertext=[ 'DLHDRXHLLDLBRXHLLKSMFVRDNHXXVDLRWFHAYCHDLCYAKXHMKGXXHMFQKMRKNDODNR
SYIXKCFYIKMYXZKAMRKNODOKAIVFQKDLDFDHXMFHFKHLABYHXDLNYLFSHMFFYFQYMKDLGQDNFQKSKD
MLYDLRWFAYCHDLMRKNDODKAMWNQRXHLLKSMHSKNHXXKAAYCHDLDLAKRKLAKLFFYKCRQHMDFQKOHNFQ
HFFQKVNHLMYXZKRXHLLDLBRSYIXKCMOSYCHGDAKSHLBKYOAYCHDLMFVRDNHXXKPHCRXKMYOAYCHDLMS
KIXYNUFMHNUDLBXYBDMFDMGYSUOXYGCHLHBKCKLFHLASYIYFFHMURXHLLDLBQKLNKHMDBXKAYCHDL
DLAKRKLAKLFRXHLLKSNHLIKWMKAFYMYXZKRXHLLDLBRSYIXKCMDLHXXFQKMKZHSDYWMA YCHDLYLFQKY
FQKSQHLAHSYWFKRXHLLKSDMFVRDNHXYOHAYCHDLMRKNDODNRXHLLKS' ]';
tabulate(ciphertext)
```

使用C语言统计单, 两, 三字母组合的频率, 代码如下:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// 你的密文 (直接粘进去)
const char* ciphertext =
"DLHDRXHLLDLBRXHLLKSMFVRDNHXXVDLRWFHAYCHDLCYAKXHMKGXXHMFQKMRKNDODNRSYIXKCFYIKMY
XZKAMRKNODOKAIVFQKDLDFDHXMFHFKHLABYHXDLNYLFSHMFFYFQYMKDLGQDNFQKSKDMLYDLRWFAYCH
DLMRKNDODKAMWNQRXHLLKSMHSKNHXXKAAYCHDLDLAKRKLAKLFFYKCRQHMDFQKOHNFQFHFFQKVNHLMYX
ZKRXHLLDLBRSYIXKCMOSYCHGDAKSHLBKYOAYCHDLMFVRDNHXXKPHCRXKMYOAYCHDLMHSKIXYNUFMHNU
LBXYBDMFDMGYSUOXYGCHLHBKCKLFHLASYIYFFHMURXHLLDLBQKLNKHMDBXKAYCHDLDLAKRKLAKLFR
XHLLKSNHLIKWMKAFYMYXZKRXHLLDLBRSYIXKCMDLHXXFQKMKZHSDYWMA YCHDLYLFQKYFQKSQHLAHSY
WFKRXHLLKSDMFVRDNHXYOHAYCHDLMRKNDODNRXHLLKS";

int main() {
    int len = strlen(ciphertext);

    // 1. 单个字母计数
    int letter[26] = { 0 };
```

```

// 2. bigram 计数 (26×26)
int bigram[26][26] = { {0} };

// 3. trigram 计数 (26×26×26 = 17 576) 用一维数组模拟三维
int trigram[26 * 26 * 26] = { 0 };

// 统计
for (int i = 0; i < len; i++) {
    char c = toupper(ciphertext[i]);
    if (c < 'A' || c > 'Z') continue;

    int a = c - 'A';
    letter[a]++;

    // bigram
    if (i + 1 < len) {
        char c2 = toupper(ciphertext[i + 1]);
        if (c2 >= 'A' && c2 <= 'Z') {
            int b = c2 - 'A';
            bigram[a][b]++;
        }
    }

    // trigram
    if (i + 2 < len) {
        char c2 = toupper(ciphertext[i + 1]);
        char c3 = toupper(ciphertext[i + 2]);
        if (c2 >= 'A' && c2 <= 'Z' && c3 >= 'A' && c3 <= 'Z') {
            int idx = a * 676 + (c2 - 'A') * 26 + (c3 - 'A'); // 26^2=676
            trigram[idx]++;
        }
    }
}

// ===== 输出 单个字母 =====
printf("=== 单个字母出现次数 ===\n");
for (int i = 0; i < 26; i++) {
    if (letter[i] > 0)
        printf("%c: %d\n", 'A' + i, letter[i]);
}
printf("\n");

// ===== 输出 Bigram (前50个最常见) =====

```

```

printf("=== Bigram 相邻两个字母组合 (前50个) ===\n");
struct Item2 { char a, b; int cnt; } list2[676];
int n2 = 0;
for (int i = 0; i < 26; i++)
    for (int j = 0; j < 26; j++)
        if (bigram[i][j]) {
            list2[n2].a = 'A' + i;
            list2[n2].b = 'A' + j;
            list2[n2].cnt = bigram[i][j];
            n2++;
        }

// 降序排序
for (int i = 0; i < n2 - 1; i++)
    for (int j = 0; j < n2 - i - 1; j++)
        if (list2[j].cnt < list2[j + 1].cnt) {
            struct Item2 t = list2[j];
            list2[j] = list2[j + 1];
            list2[j + 1] = t;
        }

for (int i = 0; i < n2 && i < 50; i++)
    printf("%c%c : %d\n", list2[i].a, list2[i].b, list2[i].cnt);
printf("\n");

// ===== 输出 Trigram (前50个最常见) =====
printf("=== Trigram 相邻三个字母组合 (前50个) ===\n");
struct Item3 { char a, b, c; int cnt; } list3[17576];
int n3 = 0;
for (int idx = 0; idx < 26 * 26 * 26; idx++) {
    if (trigram[idx] == 0) continue;
    int a = idx / 676;
    int b = (idx / 26) % 26;
    int c = idx % 26;
    list3[n3].a = 'A' + a;
    list3[n3].b = 'A' + b;
    list3[n3].c = 'A' + c;
    list3[n3].cnt = trigram[idx];
    n3++;
}

// 降序排序
for (int i = 0; i < n3 - 1; i++)
    for (int j = 0; j < n3 - i - 1; j++)

```

```
        if (list3[j].cnt < list3[j + 1].cnt) {
            struct Item3 t = list3[j];
            list3[j] = list3[j + 1];
            list3[j + 1] = t;
        }

    for (int i = 0; i < n3 && i < 50; i++)
        printf("%c%c%c : %d\n", list3[i].a, list3[i].b, list3[i].c,
list3[i].cnt);

    return 0;
}结果如下:
```

DL : 23	DO : 4	LAK : 4	=== 单个
HL : 16	FY : 4	LLD : 4	A: 23
XH : 11	HS : 4	MRK : 4	B: 10
CH : 10	IX : 4	NDO : 4	C: 17
FQ : 10	KM : 4	NHX : 4	D: 45
RX : 10	MR : 4	RKN : 4	F: 34
HD : 9	MY : 4	SYI : 4	G: 5
LL : 9	ND : 4	DLD : 3	H: 51
QK : 9	OD : 4	DNH : 3	I: 8
YC : 9	XX : 4	FVR : 3	K: 54
AY : 8	XY : 4	HLA : 3	L: 57
KS : 8	BR : 3	HXX : 3	M: 31
MF : 8	DM : 3	IXK : 3	N: 20
DN : 7	FV : 3	KLF : 3	O: 10
HX : 7	KD : 3	KRX : 3	P: 1
LA : 7		LBR : 3	Q: 15
LB : 7	=== Trig	MFV : 3	R: 25
LD : 7	HLL : 9	MYX : 3	S: 19
XK : 7	RXH : 9	RDN : 3	U: 4
AK : 6	XHL : 9	RSY : 3	V: 6
HM : 6	YCH : 9	VRD : 3	W: 6
KA : 6	AYC : 8	XKC : 3	X: 33
KL : 6	CHD : 8	XZK : 3	Y: 37
NH : 6	FQK : 8	YIX : 3	Z: 4
RK : 6	HDL : 8	YXZ : 3	
SY : 6	DLB : 6	AKL : 2	=== Bigra
FF : 5	LDL : 6	AKR : 2	DL : 23
FH : 5	LKS : 5	BRS : 2	HL : 16
KC : 5	LLK : 5	DKA : 2	XH : 11
KN : 5	DLM : 4	DLA : 2	CH : 10
KR : 5	DOD : 4	DLH : 2	FQ : 10
LF : 5	KND : 4	DLR : 2	RX : 10
LK : 5	LAK : 4	DMF : 2	HD : 9
LM : 5	LLD : 4	DNR : 2	LL : 9
YI : 5	MRK : 4	FFQ : 2	QK : 9
DO : 4			YC : 9

单字符出现率最高的是L和K，可能对应e和t，出现率最低的是P，可能对应z或x。经过尝试发现K对应t或P对应z无法得到有意义的解，故将K对应e，P对应x，替换后得到：

三字符出现率高的几个组合需要注意，可能是英语中常见的the等单词

DLHDXHLLDLBRXHLLLeSMFVRDNHXXVDLRWFHAYCHDLCYAeXHMGeXXHMFQeMReNDODNRSYIXeCFYIeMYXZeAMReNDODeAIVFQeDLDFDHXMFHFeHLABYHXDLNLYFSHMFYFQYMeDLGQDNAFQeSeDMLYDLRWFAYCHDLMRenDODeAMWNQRXHLLLeSMHSeNHXXeAAYCHDLDLAeReLAeLFFYeCRQHMDfQeOHNFFQHFFQeVNHLMYXZeRXHLLDLBRSYIXeCMOSYCHGDAeSHLBeYOAYCHDLMFVRDNHxePHCRXeMYOAYCHDLMHSeIXYNUMFHNUDLBXYBDMFDNMGYUOXYGCHLHBeCeLFHLASYIYFFHMURXHLLDLBQeLNeHMDLBXeAYCHDLDLAeReLAeLFRXHLLLeSNHLIeWMeAFYMYXZeRXHLLDLBRSYIXeCMDLHXXFQeMeZHSYWMAYCHDLYLFQeYFQeSQHLAHSYWFeRXHLLLeSDMFVRDNHXYOHAYCHDLMRenDODNRXHLLLeS

注意到：AYCHDL，RDXLLeS，RDXLLDLB经常出现，需要注意拆分后得到：

DLHD RXHLLDLB RXHLLLeS MFVRDNHXXVDLRWFH AYCHDL
CYAeXHMGeXXHMFQeMReNDODNRSYIXeCFYIeMYXZeAMReNDODeAIVFQeDLDFDHXMFHFeHLABYHXDLNLYFSHMFYFQYMeDLGQDNAFQeSeDMLYDLRWF AYCHDL MReNDODeAMWNQ RXHLLLeS MHSeNHXXeA AYCHDL
DLAeReLAeLFFYeCRQHMDfQeOHNFFQHFFQeVNHLMYXZe RXHLLDLB RSYIXeCMOSYCHGDAeSHLBeYOAYCHDL MFVRDNHxePHCRXeMYO AYCHDL
MHSeIXYNUMFHNUDLBXYBDMFDNMGYUOXYGCHLHBeCeLFHLASYIYFFHMURXHLLDLB QeLNeHMDLBXeAYCHDL DLAeReLAeLFRXHLLLeS NHLeWMeAFYMYXZe RXHLLDLB RSYIXeCMDLHXXFQeMeZHSYWMAYCHDL YLFQeYFQeSQHLAHSYWFe RXHLLLeS DMFVRDNHXYOH AYCHDL MReNDODNRXHLLLeS

DL出现概率最高，替换为th或he经数次尝试均无进展，故替换为in，得到：

inHi RXHnninB RXHnneS MFVRiNHXXVinRWFH AYCHin
CYAeXHMGeXXHMFQeMReNiOinRSYIXeCFYIeMYXZeAMReNiOieAIVFQeiniFiHXMFHFeHnABYHXinNynFSHMFYFQYMeinGQinAFQeSeiMnYinRWF AYCHin MReNiOieAMWNQ RXHnneS MHSeNHXXeA AYCHin
inAeRenAenFFYeCRQHMiFQeOHNFFQHFFQeVNHNMYXZe RXHnninB RSYIXeCMOSYCHGiAeShnBeYOAYCHin MFVRiNHxePHCRXeMYO AYCHin
MHSeIXYNUMFHNUinBXYBiMFiNMGYUOXYGCHnHBecenFhnASyIYFFHMURXHnninB QenNeHMinBXeAYCHin inAeRenAenF RXHnneS NHLeWMeAFYMYXZe RXHnninB RSYIXeCMinHXXFQeMeZHSiYWMAYCHin YnFQeYFQeSQHnAHSYWFe RXHnneS iMFVRiNHXYOH AYCHin MReNiOin RXHnneS

RXHnninB 形似 planning，替换后可得：

inai planning planneS MFVpiNallVinpWFa AYCain
CYAelaMGellaMFQeMpeNiOinPSYIleCFYIeMYlZeAMpeNiOieAIVFQeiniFialMFaFeanAgYalinNynFSaMFFYFQYMeinGQinAFQeSeiMnYinpWF AYCain MpeNiOieAMWNQ planneS MaSeNalleA AYCain
inAepenAenFFYeCpQaMiFQeOaNFFQaFFQeVNanMYlZe planning pSYIleCMOSYCaGiAeSangeYOAYCain MFVpiNalePaCpleMYO AYCain
MaSeIlyNUMFaUinglygiMFiNMGYUOlYGCnagCenFanASyIYFFaMU planning QenNeaMingleAYCain inAepenAenF planneS NanLeWMeAFYMYlZe planning pSYIleCMinalIFQeMeZaSiYWMAYCain YnFQeYFQeSQanAaSYWFe planneS iMFVpiNalyOa AYCain MpeNiOin planneS

planneS 形似 planner，替换后可得：

inaiplanning plannerMFVpiNallVinpWFaAYCain
CYAelaMGellaMFQeMpeNiOinPrYIleCFYIeMYlZeAMpeNiOieAIVFQeiniFialMFaFeanAgYalinNynFramFFYFQYMeinGQinAFQereiMnYinpWFAYCainMpeNiOieAMWNQplannerMareNalleAAYCaininAepenAenFFYeCpQaMiFQeOaNFFQaFFQeVNanMYlZe

planningprYIleCMOrYCaGiAerangeYOAYCainMFVpiNalePaCpleMYOAYCain
MareIlyNUMFaNUinglygiMFiNMGYrUOlyGCanageCenFanArYIYFFaMUpplanning
QenNeaMingleAYCaininAepenAenFplannerNanIeWMeAFYMYlZeplanning
prYIleCMinallFQeMeZariYWMAYCainYnFQeYFQerQanAarYWFepplanner
iMFVpiNalyOaAYCainMpeNiOiN

inAepenAenF 形似 independent, 替换后得到

inaiplanningplannerMtVpiNallVinpWtadYcainCYdelamGellaMtQeMpeNiOiNprYIleCtYIeMYlZedM
peNiOiedIVtQeinitialMtateandYalinYntraMttYtQYMeinGQiNdtQereiMnYinpWtdYcainMpeNiOi
edMWNQplannerMareNalleddYcainindependenttYeCpQaMitQeOaNttQattQeVNanMYlZeplanningprY
IleCMOrYCaGiderangeYOdYcainMtVpiNalexapCpleMYOdYcainMareIlyNUMtaNUinglygiMtiNMGYrUO
lyGCanageCentandrYIYttaMUpplanningQenNeaMingledYcainindependentplannerNanIeWMedtYMYlZ
eplanningprYIleCMinalltQeMeZariYWMdYcainYntQeYtQerQandarYWteplanneriMtVpiNalyOadYCa
inMpeNiOiNplanner

dYcain 形似 domian, 替换后得到:

inaiplanningplannerMtVpiNallVinpWtadomainmodelaMGellaMtQeMpeNiOiNproIlemtoIeMolZedM
peNiOiedIVtQeinitialMtateandgoalinNontraMttotQoMeinGQiNdtQereiMnoinpWtdomainMpeNiOi
edMWNQplannerMareNalreddomainindependenttoempQaMitQeOaNttQattQeVNanMolZeplanningpro
IlemMOramaGiderangeoOdomainMtVpiNalexampleMoOdomainMareIloNUMtaNUinglogiMtiNMGorUOl
oGmanagementandroIottaMUpplanningQenNeaMingledomainindependentplannerNanIeWMedtoMolZ
eplanningproIlemMinalltQeMeZarioWMdomainontQeotQerQandarOWteplanneriMtVpiNaloOadoma
inMpeNiOiNplanner

重新按词划分后得到, 更加清晰:

In ai planning planner MtVpiNallVinpWta domain model aMGellaM
tQeMpeNiOiNproIlemtoIeMolZedMpeNiOiedIVtQeinitialMtateandgoalinNontraMttotQoMeinGQi
NdtQereiMnoinpWtdomainMpeNiOiedMWNQ planner MareNalled domain independent
toempQaMitQeOaNttQattQeVNanMolZe planning proIlem MOramaGiderangeoO domain MtVpiNal
example MoOdomainMareIloNUMtaNUinglogiMtiNMGorUOl oG management and roIottaMU
planning QenNeaMingle domain independent planner NanIeWMedtoMolZe planning proIlem
MinalltQeMeZarioWMdomainontQeotQerQandarOWte planner iMtVpiNaloOa domain MpeNiOiN
planner

aMGellaM 形似 as well as, 替换得:

inaiplanningplannerstVpiNallVinpWtadomainmodelaswellastQespeNiOiNproIlemtoIesolZeds
peNiOiedIVtQeinitialstateandgoalinNontrasttotQoseinwQiNdtQereisnoinpWtdomainspeNiOi
edsWNQplannersareNalreddomainindependenttoempQasitQeOaNttQattQeVNansolZeplanningpro
IlemsOromawiderangeoOdomainstVpiNalexamplesoOdomainsareIloNUstaNUinglogistiNsworUOl
owmanagementandroIottasUpplanningQenNeasingledomainindependentplannerNanIeWsedtosolZ
eplanningproIlemsinalltQeseZarioWsdomainontQeotQerQandarOWteplanneristVpiNaloOadoma
inspeNiOiNplanner

proIlem 形似 problem, 替换得:

inaiplanningplannerstVpiNallVinpWtadomainmodelaswellastQespeNiOiNproblemtobesolZeds
peNiOiedbVtQeinitialstateandgoalinNontrasttotQoseinwQiNdtQereisnoinpWtdomainspeNiOi
edsWNQplannersareNalreddomainindependenttoempQasitQeOaNttQattQeVNansolZeplanningpro
blemsOromawiderangeoOdomainstVpiNalexamplesoOdomainsarebloNUstaNUinglogistiNsworUOl

ow management and robot task planning. A general domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

后续 as well as tQe, tQe 推测为 the, 替换得:

In artificial planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasize the fact that they can solve planning problems from a wide range of domains typical examples of domains are block stacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

problem to be solved solve 推测为 solve, 替换得:

In artificial planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasize the fact that they can solve planning problems from a wide range of domains typical examples of domains are block stacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

specific 形似 specific, 替换得:

In artificial planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasize the fact that they can solve planning problems from a wide range of domains typical examples of domains are block stacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

typically 为 typically, 替换得:

In artificial planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasize the fact that they can solve planning problems from a wide range of domains typical examples of domains are block stacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

input and such 推测为 input 和 such, 替换得:

In artificial planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasize the fact that they can solve planning problems from a wide range of domains typical examples of domains are block stacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domains on the other hand a route planner is typical of a domain specific planner

tasU 形似 task, worU 形似 work, 将 U 替换成 k 并整理修正得:

in ai planning planners typically input a domain model as well as the specific problem to be solved specified by the initial state and goal in contrast to those in which there is no input domain specified such planners are called domain independent to emphasis the fact that they can solve planning problems from a wide range of domains typical examples of domains are blocks tacking logistics workflow management and robot task planning hence a single domain independent planner can be used to solve planning problems in all these various domain on the other hand a route planner is typical of a domain specific planner

2、Enigma 密码编写; (输入明文长度是任意的)

例: 明文: 量子通信保密技术的诞生和快速发展主要取决于以下两个因素: a、经典保密通信面临着三个难以彻底解决的关键问题, 即密钥协商、身份识别和窃听检测, 这些问题的有效解决需要新技术。b、在对新技术的探索中, 人们发现了量子内在的安全特性及其可能的应用。

实现功能: 使用了 3 个转子, 具有反射器, 并同时具有加密和解密功能。

将汉字转化为拼音后大写, 得到:

LIANGZITONGXINBAOMIJISHUDEDANSHENGHEKUAISUFANZHANZHUYAOQUJUEYUYIXIALIANGGEYINSUJ
INGDIANBAOMITONGXINMIANLINZHESANGENANYICHEDIJIEJUEDEGUANJIANWENTIJIMIYAOXIESHAN
GSHEFENSHIBIEHEQIETINGJIANCEZHEXIEWENTIDEYOUXIAOJIEJUEXUYAOXINJISHUZAIIDUIXINJI
SHUDETANSUOZHONGRENMENFAXIANLELIANGZINEIZAIDEANQUANTEXINGJIQIKENENGDEYINGYONG

使用 matlab 命令如下:

```
function enigma_machine()
% 定义转子和反射器
rotor1 = 'EKMFLGDQVZNTOWYHXUSPAIBRCJ'; % I
rotor2 = 'AJDKSIRUXBLHWTMCQGZNPYFVOE'; % II
rotor3 = 'BDFHJLCPRXTVZNYEIWGAKMUSQO'; % III
reflector = 'YRUHQSLDPXNGOKMIEBFZCWVJAT'; % Reflector B

plaintext = input('请输入明文 (仅限大写字母): ', 's');
plaintext = upper(plaintext); % 转换为大写

% 设置转子初始位置 (环设置), 这里都设置为'A'
rotor1_pos = 1; % A
rotor2_pos = 1; % A
rotor3_pos = 1; % A
```

```

% 加密
ciphertext = '';
for i = 1:length(plaintext)
    char_index = plaintext(i) - 'A' + 1;

    % 转子转动 (步进)
    rotor1_pos = mod(rotor1_pos, 26) + 1;
    if rotor1_pos == 1
        rotor2_pos = mod(rotor2_pos, 26) + 1;
        if rotor2_pos == 1
            rotor3_pos = mod(rotor3_pos, 26) + 1;
        end
    end
end

char_index = rotor_pass(char_index, rotor1, rotor1_pos);
char_index = rotor_pass(char_index, rotor2, rotor2_pos);
char_index = rotor_pass(char_index, rotor3, rotor3_pos);

char_index = reflector(char_index) - 'A' + 1;

char_index = reverse_rotor_pass(char_index, rotor3, rotor3_pos);
char_index = reverse_rotor_pass(char_index, rotor2, rotor2_pos);
char_index = reverse_rotor_pass(char_index, rotor1, rotor1_pos);

ciphertext = [ciphertext, char(char_index + 'A' - 1)];
end

% 解密
decryptedtext = '';
rotor1_pos = 1; % A
rotor2_pos = 1; % A
rotor3_pos = 1; % A
for i = 1:length(ciphertext)
    char_index = ciphertext(i) - 'A' + 1;

    % 转子转动
    rotor1_pos = mod(rotor1_pos, 26) + 1;
    if rotor1_pos == 1
        rotor2_pos = mod(rotor2_pos, 26) + 1;
        if rotor2_pos == 1
            rotor3_pos = mod(rotor3_pos, 26) + 1;
        end
    end
end

```

```

%解密
char_index = rotor_pass(char_index, rotor1, rotor1_pos);
char_index = rotor_pass(char_index, rotor2, rotor2_pos);
char_index = rotor_pass(char_index, rotor3, rotor3_pos);
char_index = reflector(char_index) - 'A' + 1;
char_index = reverse_rotor_pass(char_index, rotor3, rotor3_pos);
char_index = reverse_rotor_pass(char_index, rotor2, rotor2_pos);
char_index = reverse_rotor_pass(char_index, rotor1, rotor1_pos);

decryptedtext = [decryptedtext, char(char_index + 'A' - 1)];
end

% 输出结果
disp(['密文: ', ciphertext]);
disp(['解密后: ', decryptedtext]);
end

% 转子正向传递
function output = rotor_pass(input, rotor, rotor_pos)
    output = rotor(mod(input + rotor_pos - 2, 26) + 1) - 'A' + 1; %包含偏移的转子加密
end

% 转子反向传递
function output = reverse_rotor_pass(input, rotor, rotor_pos)
    shifted_rotor = circshift(rotor, -rotor_pos + 1); % 将转子根据位置旋转
    output = find(shifted_rotor == char(input + 'A' - 1));
end

```

按照以上命令输出得到:

```

>> enigma_machine
请输入明文 (仅限大写字母): LIANGZITONGXINBAOMIJISHUDEDANSHENGHEKUAISUFZAHANZHUYAOQUJUEYUYIXIALIANGGEYINSUJINGDIANBAOMITONGXINMIANLINZHE SANGENANYICHEDI
密文: PFVPKCEGEDIGRAXWRDVEDFYLVZSKTNVQEOAHDJULGUTQQLAKHOVTJAAHZDBZBWDRYOGWZTXGR IYCFEECWPJGXWGHBCPOVNPVWHPYHBWDGUJTAPDELUJXNYCBMWVWCCRQFSBTKMLMDVHBS
解密后: LIANGZITONGXINBAOMIJISHUDEDANSHENGHEKUAISUFZAHANZHUYAOQUJUEYUYIXIALIANGGEYINSUJINGDIANBAOMITONGXINMIANLINZHE SANGENANYICHEDIJIEJUEDEGUANJIANWENTI
JIMIYAOXIESHANGSHENFENSHIBIEHEQIETINGJIANCEZHEXIEWENTIDEYOUXIAOJIEJUEXUYAOXINJISH
UZAUZIDUOXINJISHUDETANSUOZHONGRENMENFAXIANLELIANGZINEIZAIIDEANQUANTEXINGJIQIKE
NENGDEYINGYONG

```

```

>> enigma
请输入明文 (仅限大写字母):
LIANGZITONGXINBAOMIJISHUDEDANSHENGHEKUAISUFZAHANZHUYAOQUJUEYUYIXIALIANGGEYI
NSUJINGDIANBAOMITONGXINMIANLINZHE SANGENANYICHEDIJIEJUEDEGUANJIANWENTIJIMIYA
OXIESHANGSHENFENSHIBIEHEQIETINGJIANCEZHEXIEWENTIDEYOUXIAOJIEJUEXUYAOXINJISH
UZAUZIDUOXINJISHUDETANSUOZHONGRENMENFAXIANLELIANGZINEIZAIIDEANQUANTEXINGJIQIKE
NENGDEYINGYONG
密文:
PFVPKCEGEDIGRAXWRDVEDFYLVZSKTNVQEOAHDJULGUTQQLAKHOVTJAAHZDBZBWDRYOGWZTXGR
IYCFEECWPJGXWGHBCPOVNPVWHPYHBWDGUJTAPDELUJXNYCBMWVWCCRQFSBTKMLMDVHBSYVUZWIO

```

APJJCCUGTFMMGMLBNEOUWXRDKFMJZHWYCPUUCMHPGBTHMQQAFBLRSLIMZXKDFTFTEPZTQOEDOS
OEIQTWOIEXHGDQHZNGWDHJBFBMLXPVORWDXOTLYUPCRJQRHXQWZIRLGVJYFMQWLDVQLKGGTG
LCXVVSZMXHWGP

解密后:

LIANGZITONGXINBAOMIJISHUDEDANSHENGHEKUAISUFAZHANZHUYAOQUJUEYUYIXIALIANGGEYI
NSUJINGDIANBAOMITONGXINMIANLINZHESANGENANYICHEDIJIEJUEDEGUANJIANWENTIJI MIYA
OXIESHANGSHENFENSHIBIEHEQIETINGJIANCEZHEXIEWENTIDEYOUXIAOJIEJUEXUYAOXINJISH
UZAUDEXINJISHUDETANSUOZHONGRENMENFAXIANLELIANGZINEIZAIDEANQUANTEXINGJIQIKE
NENGDEYINGYONG

3、AES 加密、解密算法的编写;

- (1)、最低要求: 按照 AES 要求, 编写一固定长度明文的加密;
 - (2)、一般要求: 可以实现任意定长密文的解密;
 - (3)、高要求: 明密文长度不限, 并利用某一迭代方式 (非分段后, 并行处理) 加解密。
- 基本要求: 画出算法结构图, 表述各函数功能。

实现功能: 高要求, 明文长度任意, 密钥长度固定为 16 字符, 支持解密功能。

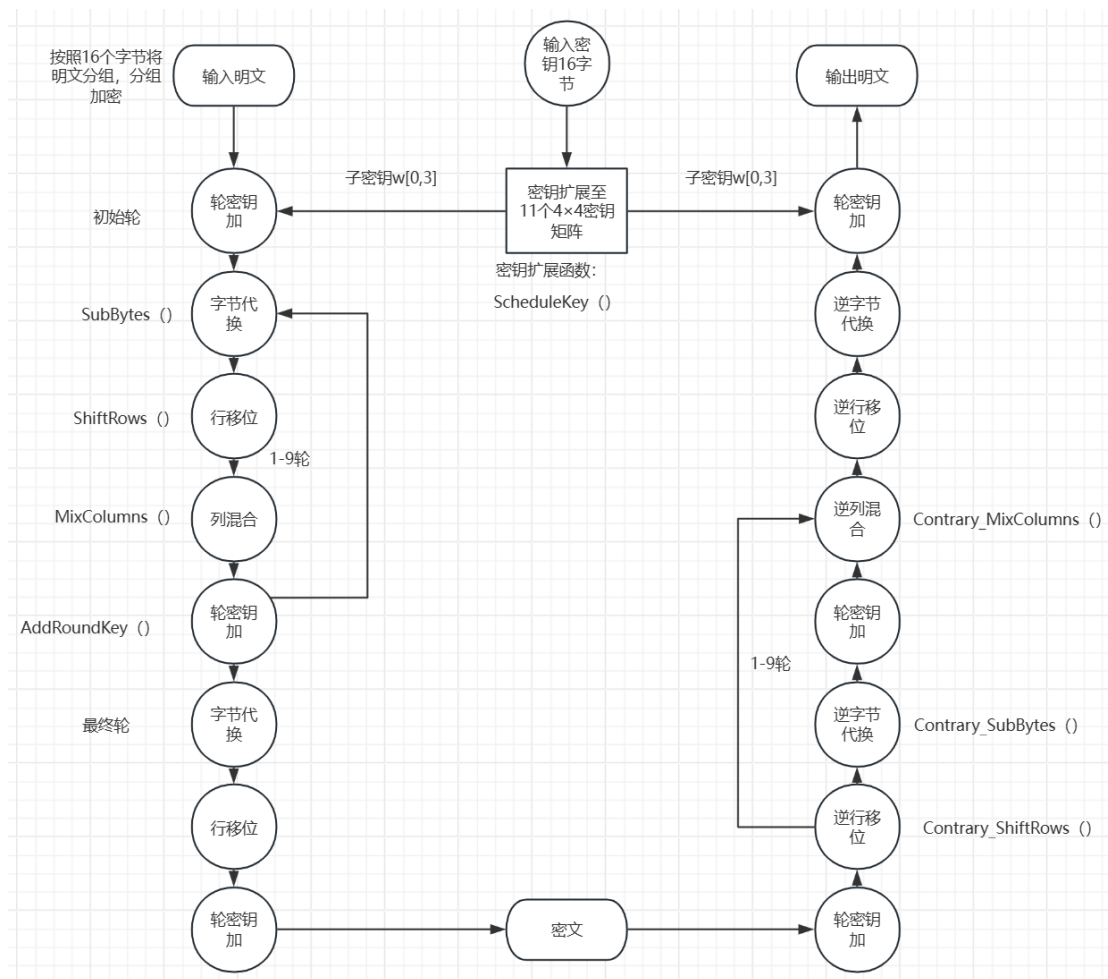
AES 有 4 个关键步骤, 如下所示:

- (1) 首先是 SubBytes (字节代换), 将每个字节的值按照 S 盒替换成另外的值;
 - (2) 其次是 ShiftRows (行移位), 将明文以 4 个字节为一行, 转化为多行, 也就是矩阵, 每行按照一定规则向左平移;
 - (3) 然后是 MixColumns (列混合), 将矩阵的列, 每 4 个字节为单位进行比特运算, 转化为另外的 4 个字节值;
 - (4) 最后是 AddRoundKey (轮密钥加), 将 MixColumns 后的数据与轮密钥进行 XOR 运算。
- 这是一轮的操作。解密则是相反的顺序。

在 AES-128 中, 密钥和分组都是 128 位 (16 字节), 加密过程分为:

- 1. 密钥扩展
- 2. 初始轮: 轮密钥加
- 3. 主轮 1-9: 字节代换, 行移位, 列混合, 轮密钥加
- 4. 最终轮: 字节代换, 行移位, 轮密钥加

流程图如下



代码如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#pragma warning(disable:4996) // 禁用 VS 的 4996 警告 (兼容 fgets/scanf 等函数)
```

// ----- 基础常量定义 -----

// Rcon: 密钥扩展时使用的轮常量 (AES-128 共 10 轮, 对应 10 个常量)

// 作用: 确保每轮扩展的密钥不同, 提升安全性

```
static const unsigned char Rcon[10] = {
    0x01, 0x02, 0x04, 0x08, 0x10,
    0x20, 0x40, 0x80, 0x1b, 0x36
};
```

// S 盒 (Substitution Box): 正向替换表 (加密时使用)

// 本质: 256 字节的固定非线性置换, 通过仿射变换+有限域求逆生成

// 作用: 对单个字节进行替换, 引入非线性, 抵抗线性攻击

```
static const unsigned char sbox[256] = {
    0x63, 0x7c, 0x77, 0x7b, 0xf2, 0x6b, 0x6f, 0xc5, 0x30, 0x01, 0x67,
```

```

0x2b, 0xfe, 0xd7, 0xab, 0x76,
    0xca, 0x82, 0xc9, 0x7d, 0xfa, 0x59, 0x47, 0xf0, 0xad, 0xd4, 0xa2,
0xaf, 0x9c, 0xa4, 0x72, 0xc0,
    0xb7, 0xfd, 0x93, 0x26, 0x36, 0x3f, 0xf7, 0xcc, 0x34, 0xa5, 0xe5,
0xf1, 0x71, 0xd8, 0x31, 0x15,
    0x04, 0xc7, 0x23, 0xc3, 0x18, 0x96, 0x05, 0x9a, 0x07, 0x12, 0x80,
0xe2, 0xeb, 0x27, 0xb2, 0x75,
    0x09, 0x83, 0x2c, 0x1a, 0x1b, 0x6e, 0x5a, 0xa0, 0x52, 0x3b, 0xd6,
0xb3, 0x29, 0xe3, 0x2f, 0x84,
    0x53, 0xd1, 0x00, 0xed, 0x20, 0xfc, 0xb1, 0x5b, 0x6a, 0xcb, 0xbe,
0x39, 0x4a, 0x4c, 0x58, 0xcf,
    0xd0, 0xef, 0xaa, 0xfb, 0x43, 0x4d, 0x33, 0x85, 0x45, 0xf9, 0x02,
0x7f, 0x50, 0x3c, 0x9f, 0xa8,
    0x51, 0xa3, 0x40, 0x8f, 0x92, 0x9d, 0x38, 0xf5, 0xbc, 0xb6, 0xda,
0x21, 0x10, 0xff, 0xf3, 0xd2,
    0xcd, 0x0c, 0x13, 0xec, 0x5f, 0x97, 0x44, 0x17, 0xc4, 0xa7, 0x7e,
0x3d, 0x64, 0x5d, 0x19, 0x73,
    0x60, 0x81, 0x4f, 0xdc, 0x22, 0x2a, 0x90, 0x88, 0x46, 0xee, 0xb8,
0x14, 0xde, 0x5e, 0x0b, 0xdb,
    0xe0, 0x32, 0x3a, 0x0a, 0x49, 0x06, 0x24, 0x5c, 0xc2, 0xd3, 0xac,
0x62, 0x91, 0x95, 0xe4, 0x79,
    0xe7, 0xc8, 0x37, 0x6d, 0x8d, 0xd5, 0x4e, 0xa9, 0x6c, 0x56, 0xf4,
0xea, 0x65, 0x7a, 0xae, 0x08,
    0xba, 0x78, 0x25, 0x2e, 0x1c, 0xa6, 0xb4, 0xc6, 0xe8, 0xdd, 0x74,
0x1f, 0x4b, 0xbd, 0x8b, 0x8a,
    0x70, 0x3e, 0xb5, 0x66, 0x48, 0x03, 0xf6, 0x0e, 0x61, 0x35, 0x57,
0xb9, 0x86, 0xc1, 0x1d, 0x9e,
    0xe1, 0xf8, 0x98, 0x11, 0x69, 0xd9, 0x8e, 0x94, 0x9b, 0x1e, 0x87,
0xe9, 0xce, 0x55, 0x28, 0xdf,
    0x8c, 0xa1, 0x89, 0x0d, 0xbf, 0xe6, 0x42, 0x68, 0x41, 0x99, 0x2d,
0x0f, 0xb0, 0x54, 0xbb, 0x16
};

```

// 逆S盒 (Contrary Substitution Box): 解密时使用

// 作用: S盒的逆映射, 还原加密时的字节替换

```

static const unsigned char contrary_sbox[256] = {
    0x52, 0x09, 0x6a, 0xd5, 0x30, 0x36, 0xa5, 0x38, 0xbf, 0x40, 0xa3,
0x9e, 0x81, 0xf3, 0xd7, 0xfb,
    0x7c, 0xe3, 0x39, 0x82, 0x9b, 0x2f, 0xff, 0x87, 0x34, 0x8e, 0x43,
0x44, 0xc4, 0xde, 0xe9, 0xcb,
    0x54, 0x7b, 0x94, 0x32, 0xa6, 0xc2, 0x23, 0x3d, 0xee, 0x4c, 0x95,
0x0b, 0x42, 0xfa, 0xc3, 0x4e,
    0x08, 0x2e, 0xa1, 0x66, 0x28, 0xd9, 0x24, 0xb2, 0x76, 0x5b, 0xa2,
0x49, 0x6d, 0x8b, 0xd1, 0x25,

```

```

        0x72, 0xf8, 0xf6, 0x64, 0x86, 0x68, 0x98, 0x16, 0xd4, 0xa4, 0x5c,
0xcc, 0x5d, 0x65, 0xb6, 0x92,
        0x6c, 0x70, 0x48, 0x50, 0xfd, 0xed, 0xb9, 0xda, 0x5e, 0x15, 0x46,
0x57, 0xa7, 0x8d, 0x9d, 0x84,
        0x90, 0xd8, 0xab, 0x00, 0x8c, 0xbc, 0xd3, 0x0a, 0xf7, 0xe4, 0x58,
0x05, 0xb8, 0xb3, 0x45, 0x06,
        0xd0, 0x2c, 0x1e, 0x8f, 0xca, 0x3f, 0x0f, 0x02, 0xc1, 0xaf, 0xbd,
0x03, 0x01, 0x13, 0x8a, 0x6b,
        0x3a, 0x91, 0x11, 0x41, 0x4f, 0x67, 0xdc, 0xea, 0x97, 0xf2, 0xcf,
0xce, 0xf0, 0xb4, 0xe6, 0x73,
        0x96, 0xac, 0x74, 0x22, 0xe7, 0xad, 0x35, 0x85, 0xe2, 0xf9, 0x37,
0xe8, 0x1c, 0x75, 0xdf, 0x6e,
        0x47, 0xf1, 0x1a, 0x71, 0x1d, 0x29, 0xc5, 0x89, 0x6f, 0xb7, 0x62,
0x0e, 0xaa, 0x18, 0xbe, 0x1b,
        0xfc, 0x56, 0x3e, 0x4b, 0xc6, 0xd2, 0x79, 0x20, 0x9a, 0xdb, 0xc0,
0xfe, 0x78, 0xcd, 0x5a, 0xf4,
        0x1f, 0xdd, 0xa8, 0x33, 0x88, 0x07, 0xc7, 0x31, 0xb1, 0x12, 0x10,
0x59, 0x27, 0x80, 0xec, 0x5f,
        0x60, 0x51, 0x7f, 0xa9, 0x19, 0xb5, 0x4a, 0x0d, 0x2d, 0xe5, 0x7a,
0x9f, 0x93, 0xc9, 0x9c, 0xef,
        0xa0, 0xe0, 0x3b, 0x4d, 0xae, 0x2a, 0xf5, 0xb0, 0xc8, 0xeb, 0xbb,
0x3c, 0x83, 0x53, 0x99, 0x61,
        0x17, 0x2b, 0x04, 0x7e, 0xba, 0x77, 0xd6, 0x26, 0xe1, 0x69, 0x14,
0x63, 0x55, 0x21, 0x0c, 0x7d
    };

```

// ----- 有限域乘法工具函数 -----

```

// GF(2^8)有限域乘法:  $x * 2$  (左移 1 位, 模 0x1B)
// 说明: AES 列混合操作的基础, 有限域模多项式为  $x^8+x^4+x^3+x+1$  (0x1B)
static unsigned char x2time(unsigned char x) {
    if (x & 0x80) { // 若最高位为 1 ( $x \geq 0x80$ ), 左移后会溢出, 需异或模值
0x1B
        return ((x << 1) ^ 0x1B) & 0xFF; // 左移 1 位 + 模运算, 确保 8
位
    }
    return x << 1; // 最高位为 0, 直接左移 1 位
}

// GF(2^8)有限域乘法:  $x * 3 = (x*2) \text{ XOR } x$ 
static unsigned char x3time(unsigned char x) {
    return (x2time(x) ^ x);
}

```

```

// GF(2^8)有限域乘法:  $x * 4 = (x*2)*2$ 
static unsigned char x4time(unsigned char x) {
    return (x2time(x2time(x)));
}

// GF(2^8)有限域乘法:  $x * 8 = (x*2)*2*2$ 
static unsigned char x8time(unsigned char x) {
    return (x2time(x2time(x2time(x))));
}

// GF(2^8)有限域乘法:  $x * 9 = (x*8) \text{ XOR } x$ 
static unsigned char x9time(unsigned char x) {
    return (x8time(x) ^ x);
}

// GF(2^8)有限域乘法:  $x * 11 = (x*8) \text{ XOR } (x*2) \text{ XOR } x$ 
static unsigned char xBtime(unsigned char x) {
    return (x8time(x) ^ x2time(x) ^ x);
}

// GF(2^8)有限域乘法:  $x * 13 = (x*8) \text{ XOR } (x*4) \text{ XOR } x$ 
static unsigned char xDtime(unsigned char x) {
    return (x8time(x) ^ x4time(x) ^ x);
}

// GF(2^8)有限域乘法:  $x * 14 = (x*8) \text{ XOR } (x*4) \text{ XOR } (x*2)$ 
static unsigned char xEtime(unsigned char x) {
    return (x8time(x) ^ x4time(x) ^ x2time(x));
}

// ----- AES 核心变换函数（加密） -----
// -----

// 列混合 (MixColumns): 加密时对 16 字节数据块的列进行线性变换
// 参数: col - 指向 16 字节数据块的指针（按列存储）
// 原理: 每列 4 个字节通过有限域乘法和异或重新组合，扩散数据
static void MixColumns(unsigned char* col) {
    unsigned char tmp[4]; // 临时存储变换后的列数据
    int i;
    // 遍历 4 列（每列 4 字节，数据块共 16 字节=4 列×4 行）
    for (i = 0; i < 4; i++, col += 4) {
        // 列混合公式 (AES 标准): 每列字节按 [2, 3, 1, 1] 等系数混合
        tmp[0] = x2time(col[0]) ^ x3time(col[1]) ^ col[2] ^ col[3];
        tmp[1] = col[0] ^ x2time(col[1]) ^ x3time(col[2]) ^ col[3];
        tmp[2] = col[0] ^ col[1] ^ x2time(col[2]) ^ x3time(col[3]);
    }
}

```

```

        tmp[3] = x3time(col[0]) ^ col[1] ^ col[2] ^ x2time(col[3]);
        // 将变换结果写回原数据块
        col[0] = tmp[0];
        col[1] = tmp[1];
        col[2] = tmp[2];
        col[3] = tmp[3];
    }
}

```

// 行移位 (ShiftRows): 加密时对数据块的行进行循环左移
 // 参数: col - 指向 16 字节数据块的指针 (按行存储: 第 0 行[0, 4, 8, 12], 第 1 行[1, 5, 9, 13]等)

// 原理: 第 0 行不移位, 第 1 行左移 1 位, 第 2 行左移 2 位, 第 3 行左移 3 位, 扩散数据

```

static void ShiftRows(unsigned char* col) {
    unsigned char t; // 临时变量用于交换字节

    // 第 1 行 (索引 1, 5, 9, 13): 左移 1 位
    t = col[1];
    col[1] = col[5];
    col[5] = col[9];
    col[9] = col[13];
    col[13] = t;

    // 第 2 行 (索引 2, 6, 10, 14): 左移 2 位
    t = col[2];
    col[2] = col[10];
    col[10] = t;
    t = col[6];
    col[6] = col[14];
    col[14] = t;

    // 第 3 行 (索引 3, 7, 11, 15): 左移 3 位 (等价于右移 1 位)
    t = col[15];
    col[15] = col[11];
    col[11] = col[7];
    col[7] = col[3];
    col[3] = t;
}

```

// 字节替换 (SubBytes): 加密时用 S 盒替换数据块中所有字节

// 参数: col - 指向 16 字节数据块的指针

// 作用: 引入非线性变换, 使明文与密文的关系更复杂

```

static void SubBytes(unsigned char* col) {

```

```

        int x;
        for (x = 0; x < 16; x++) {
            col[x] = sbox[col[x]]; // 逐个字节替换
        }
    }

    // ----- AES 核心变换函数（解密） -----
    // 逆列混合（Contrary_MixColumns）：解密时还原列混合操作
    // 参数：col - 指向 16 字节数据块的指针
    // 原理：使用列混合的逆变换公式，系数为[14, 11, 13, 9]等
    static void Contrary_MixColumns(unsigned char* col) {
        unsigned char tmp[4]; // 临时存储逆变换后的列数据
        int x;
        // 遍历 4 列
        for (x = 0; x < 4; x++, col += 4) {
            // 逆列混合公式（AES 标准）
            tmp[0] = xEtime(col[0]) ^ xBtime(col[1]) ^ xDtime(col[2]) ^
x9time(col[3]);
            tmp[1] = x9time(col[0]) ^ xEtime(col[1]) ^ xBtime(col[2]) ^
xDtime(col[3]);
            tmp[2] = xDtime(col[0]) ^ x9time(col[1]) ^ xEtime(col[2]) ^
xBtime(col[3]);
            tmp[3] = xBtime(col[0]) ^ xDtime(col[1]) ^ x9time(col[2]) ^
xEtime(col[3]);
            // 写回原数据块
            col[0] = tmp[0];
            col[1] = tmp[1];
            col[2] = tmp[2];
            col[3] = tmp[3];
        }
    }

    // 逆行移位（Contrary_ShiftRows）：解密时还原行移位操作
    // 参数：col - 指向 16 字节数据块的指针
    // 原理：与行移位相反：第 0 行不移位，第 1 行右移 1 位，第 2 行右移 2 位，第 3
    行右移 3 位
    static void Contrary_ShiftRows(unsigned char* col) {
        unsigned char t; // 临时变量用于交换字节

        // 第 1 行（索引 1, 5, 9, 13）：右移 1 位
        t = col[13];
        col[13] = col[9];
        col[9] = col[5];

```

```

col[5] = col[1];
col[1] = t;

// 第2行（索引 2, 6, 10, 14）：右移2位（与左移2位相同）
t = col[2];
col[2] = col[10];
col[10] = t;
t = col[6];
col[6] = col[14];
col[14] = t;

// 第3行（索引 3, 7, 11, 15）：右移3位（等价于左移1位）
t = col[3];
col[3] = col[7];
col[7] = col[11];
col[11] = col[15];
col[15] = t;
}

// 逆字节替换（Contrary_SubBytes）：解密时用逆S盒还原字节替换
// 参数：col - 指向16字节数据块的指针
static void Contrary_SubBytes(unsigned char* col) {
    int x;
    for (x = 0; x < 16; x++) {
        col[x] = contrary_sbox[col[x]]; // 逐个字节还原
    }
}

// ----- 密钥扩展函数 -----
// 密钥扩展（ScheduleKey）：将16字节原始密钥扩展为176字节轮密钥（AES-128）
// 参数：
//   inkey - 输入的16字节原始密钥（AES-128）
//   outkey - 输出的扩展密钥（11轮 × 16字节 = 176字节）
//   Nk - 密钥长度（AES-128对应Nk=4，即4个32位字）
//   Nr - 加密轮数（AES-128对应Nr=10）
// 原理：通过循环移位、S盒替换、异或Rcon，生成每轮的独立密钥
void ScheduleKey(unsigned char* inkey, unsigned char* outkey, int Nk, int Nr) {
    unsigned char temp[4]; // 临时存储4字节（1个32位字）
    int x, i;

    // 步骤1：将原始密钥复制到扩展密钥的前Nk个32位字（16字节）
    for (i = 0; i < (4 * Nk); i++) {

```

```

        outkey[i] = inkey[i];
    }

    i = Nk; // 从第 Nk 个 32 位字开始扩展
    // 步骤 2: 循环生成剩余的扩展密钥 (共 4*(Nr+1) 个 32 位字)
    while (i < (4 * (Nr + 1))) {
        // 步骤 2.1: 取上一个 32 位字作为临时变量 temp
        for (x = 0; x < 4; x++) {
            temp[x] = outkey[(4 * (i - 1)) + x];
        }

        // 步骤 2.2: 若当前是 Nk 的整数倍 (每轮第一个 32 位字), 执行特殊处
        // 理
        if (i % Nk == 0) {
            // a. 循环左移 1 字节 (temp[0]→temp[1], temp[1]→temp[2],
            // temp[3]→temp[0])
            unsigned char t = temp[0];
            temp[0] = temp[1];
            temp[1] = temp[2];
            temp[2] = temp[3];
            temp[3] = t;

            // b. S 盒替换 temp 中的每个字节
            for (x = 0; x < 4; x++) {
                temp[x] = sbox[temp[x]];
            }

            // c. 异或当前轮的 Rcon 常量 (仅第一个字节异或, 其余为 0)
            temp[0] ^= Rcon[(i / Nk) - 1];
        }

        // 步骤 2.3: 异或前 Nk 个 32 位字的对应字节, 生成当前 32 位字
        for (x = 0; x < 4; x++) {
            outkey[(4 * i) + x] = outkey[(4 * (i - Nk)) + x] ^ temp[x];
        }

        i++; // 继续生成下一个 32 位字
    }
}

// ----- 轮密钥加函数 -----
// 轮密钥加 (AddRoundKey): 将当前轮密钥与数据块逐字节异或
// 参数:
//   col - 16 字节数据块

```

```

//  expansionkey - 扩展后的轮密钥
//  round - 当前轮数 (0~Nr)
// 作用：唯一依赖密钥的操作，将密钥信息融入数据块，加密和解密逻辑相同
static void AddRoundKey(unsigned char* col, unsigned char* expansionkey,
int round) {
    int x;
    for (x = 0; x < 16; x++) {
        // 轮密钥起始位置 = 轮数 × 16 字节 (每轮 16 字节密钥)
        col[x] ^= expansionkey[(round << 4) + x];
    }
}

// ----- 顶层加密/解密函数 -----
-----

// AES 加密主函数 (AesEncrypt)：对单个 16 字节数据块进行加密
// 参数：
//  blk - 输入/输出：16 字节明文块 (加密后变为密文块)
//  expansionkey - 扩展后的轮密钥 (176 字节)
//  Nr - 加密轮数 (AES-128 对应 Nr=10)
// 流程：初始轮 (轮密钥加) → 10 轮主轮 → 最终轮 (无列混合)
void AesEncrypt(unsigned char* blk, unsigned char* expansionkey, int Nr)
{
    int round;

    // 初始轮：仅执行轮密钥加 (无字节替换、行移位、列混合)
    AddRoundKey(blk, expansionkey, 0);

    // 主轮 (1~9 轮)：字节替换 → 行移位 → 列混合 → 轮密钥加
    for (round = 1; round <= (Nr - 1); round++) {
        SubBytes(blk);
        ShiftRows(blk);
        MixColumns(blk);
        AddRoundKey(blk, expansionkey, round);
    }

    // 最终轮 (第 10 轮)：字节替换 → 行移位 → 轮密钥加 (无列混合)
    SubBytes(blk);
    ShiftRows(blk);
    AddRoundKey(blk, expansionkey, Nr);
}

// AES 解密主函数 (Contrary_AesEncrypt)：对单个 16 字节数据块进行解密
// 参数：
//  blk - 输入/输出：16 字节密文块 (解密后变为明文块)

```

```

// expansionkey - 扩展后的轮密钥 (176 字节)
// Nr - 解密轮数 (与加密轮数相同, AES-128 对应 Nr=10)
// 流程: 初始轮 (轮密钥加) → 10 轮逆主轮 → 最终逆轮
void Contrary_AesEncrypt(unsigned char* blk, unsigned char* expansionkey,
int Nr) {
    int x;

    // 初始轮: 用最后一轮密钥执行轮密钥加
    AddRoundKey(blk, expansionkey, Nr);

    // 逆行移位 → 逆字节替换 (还原最终轮的操作)
    Contrary_ShiftRows(blk);
    Contrary_SubBytes(blk);

    // 逆主轮 (9~1 轮): 轮密钥加 → 逆列混合 → 逆行移位 → 逆字节替换
    for (x = (Nr - 1); x >= 1; x--) {
        AddRoundKey(blk, expansionkey, x);
        Contrary_MixColumns(blk);
        Contrary_ShiftRows(blk);
        Contrary_SubBytes(blk);
    }

    // 最终逆轮: 用第 0 轮密钥执行轮密钥加 (还原初始轮)
    AddRoundKey(blk, expansionkey, 0);
}

// ----- 填充/去填充函数 (支持任意长度明文) -----
// PKCS#7 填充 (PKCS7Padding): 将明文补成 16 字节整数倍 (AES 分组长度)
// 参数:
//   input - 原始明文
//   input_len - 原始明文长度
//   output - 输出: 填充后的明文 (需手动释放内存)
//   output_len - 输出: 填充后的明文长度
// 规则: 填充长度 = 16 - (input_len % 16), 填充字节值 = 填充长度
void PKCS7Padding(unsigned char* input, int input_len, unsigned char**
output, int* output_len) {
    int pad_len = 16 - (input_len % 16); // 计算需要填充的字节数
    *output_len = input_len + pad_len; // 填充后的总长度
    *output = (unsigned char*)malloc(*output_len); // 分配内存
    memcpy(*output, input, input_len); // 复制原始明文

    // 填充: 从原始明文末尾开始, 填充 pad_len 个字节, 每个字节值为 pad_len
    for (int i = 0; i < pad_len; i++) {

```

```

        (*output)[input_len + i] = pad_len;
    }
}

// 移除 PKCS#7 填充 (RemovePKCS7Padding): 解密后还原原始明文
// 参数:
//   input - 填充后的明文 (解密后的结果)
//   input_len - 填充后的明文长度 (16 字节整数倍)
//   output - 输出: 原始明文 (需手动释放内存)
//   output_len - 输出: 原始明文长度
// 规则: 最后一个字节的值 = 填充长度, 截取前 (input_len - 填充长度) 个字节
void RemovePKCS7Padding(unsigned char* input, int input_len, unsigned
char** output, int* output_len) {
    int pad_len = input[input_len - 1]; // 最后一个字节是填充长度
    *output_len = input_len - pad_len; // 原始明文长度
    *output = (unsigned char*)malloc(*output_len); // 分配内存
    memcpy(*output, input, *output_len); // 复制原始明文 (去掉填充部分)
}

// ----- 支持任意长度的加密/解密函数 -----
// 任意长度明文加密 (AES_Encrypt_AnyLength): 封装填充+分块加密
// 参数:
//   plaintext - 原始明文 (任意长度)
//   plaintext_len - 原始明文长度
//   key - 16 字节加密密钥
//   ciphertext - 输出: 密文 (需手动释放内存)
//   ciphertext_len - 输出: 密文长度 (16 字节整数倍)
void AES_Encrypt_AnyLength(unsigned char* plaintext, int plaintext_len,
unsigned char* key, unsigned char** ciphertext,
int* ciphertext_len) {
    unsigned char expansionkey[15 * 16]; // 扩展密钥缓冲区 (最大支持 AES-
256, 此处用 176 字节)
    unsigned char* padded_plaintext; // 填充后的明文
    int padded_len; // 填充后的明文长度

    // 步骤 1: 密钥扩展 (AES-128: Nk=4, Nr=10)
    ScheduleKey(key, expansionkey, 4, 10);

    // 步骤 2: PKCS#7 填充, 将明文补成 16 字节整数倍
    PKCS7Padding(plaintext, plaintext_len, &padded_plaintext,
&padded_len);

```

```

// 步骤 3: 分配密文内存 (密文长度=填充后明文长度)
*ciphertext = (unsigned char*)malloc(padded_len);
*ciphertext_len = padded_len;

// 步骤 4: 分块加密 (每块 16 字节)
for (int i = 0; i < padded_len; i += 16) {
    AesEncrypt(padded_plaintext + i, expansionkey, 10); // 加密当前
块
    memcpy(*ciphertext + i, padded_plaintext + i, 16); // 复制密文
块
}

// 释放填充后的明文内存
free(padded_plaintext);
}

// 任意长度密文解密 (AES_Decrypt_AnyLength): 封装分块解密+去填充
// 参数:
//  ciphertext - 密文 (16 字节整数倍长度)
//  ciphertext_len - 密文长度
//  key - 16 字节解密密钥 (与加密密钥相同)
//  plaintext - 输出: 原始明文 (需手动释放内存)
//  plaintext_len - 输出: 原始明文长度
void AES_Decrypt_AnyLength(unsigned char* ciphertext, int ciphertext_len,
unsigned char* key, unsigned char** plaintext,
int* plaintext_len) {
    unsigned char expansionkey[15 * 16]; // 扩展密钥缓冲区
    unsigned char* padded_plaintext = (unsigned
char*)malloc(ciphertext_len); // 填充后的明文缓冲区

    // 步骤 1: 密钥扩展 (与加密时相同)
    ScheduleKey(key, expansionkey, 4, 10);

    // 步骤 2: 分块解密 (每块 16 字节)
    for (int i = 0; i < ciphertext_len; i += 16) {
        Contrary_AesEncrypt(ciphertext + i, expansionkey, 10); // 解密当
前块
        memcpy(padded_plaintext + i, ciphertext + i, 16); // 复制填
充后的明文块
    }

    // 步骤 3: 移除 PKCS#7 填充, 还原原始明文
    RemovePKCS7Padding(padded_plaintext, ciphertext_len, plaintext,
plaintext_len);
}

```

```

        // 释放填充后的明文内存
        free(padded_plaintext);
    }

    // ----- 主函数（用户交互+调用加密解密） -----
    -----

    int main(void) {
        unsigned char key[17];                // 16 字节密钥（+1 用于存储
字符串结束符）
        unsigned char plaintext[1024];        // 输入明文缓冲区（最大 1023
字节）
        unsigned char* ciphertext = NULL;     // 加密后的密文（动态分配）
        unsigned char* decrypted_plaintext = NULL; // 解密后的明文（动态分
配）
        int plaintext_len, ciphertext_len, decrypted_plaintext_len; // 各数
据长度

        printf("AES Encryption with Any Length Plaintext\n");
        printf("=====\n");

        // 步骤 1：读取用户输入的明文
        printf("Enter plaintext (up to 1023 characters):\n");
        fgets((char*)plaintext, 1024, stdin); // 读取明文（包含换行符）
        plaintext_len = strlen((char*)plaintext);
        // 去掉 fgets 读取的换行符（若存在）
        if (plaintext[plaintext_len - 1] == '\n') {
            plaintext[--plaintext_len] = '\0';
        }

        // 步骤 2：读取用户输入的 16 字节密钥
        printf("Enter 16-byte key:\n");
        scanf("%16s", key); // 限制输入 16 个字符（避免密钥过长）
        key[16] = '\0';    // 手动添加字符串结束符

        // 步骤 3：执行加密
        AES_Encrypt_AnyLength(plaintext, plaintext_len, key, &ciphertext,
&ciphertext_len);
        printf("\nCiphertext (hex):\n");
        // 以十六进制格式输出密文（每个字节占 2 位）
        for (int i = 0; i < ciphertext_len; i++) {
            printf("%02x", ciphertext[i]);
        }
        printf("\n");
    }

```

```

// 步骤 4: 执行解密
AES_Decrypt_AnyLength(ciphertext, ciphertext_len, key,
&decrypted_plaintext, &decrypted_plaintext_len);
printf("Decrypted Plaintext:\n");
// 输出解密后的明文
for (int i = 0; i < decrypted_plaintext_len; i++) {
    printf("%c", decrypted_plaintext[i]);
}
printf("\n");

// 步骤 5: 释放动态分配的内存（避免内存泄漏）
free(ciphertext);
free(decrypted_plaintext);
system("pause");
return 0;
}运行结果如下:

```

```

D:\C++\Project3\64\Debug\Project3.exe
AES Encryption with Any Length Plaintext
=====
Enter plaintext (up to 1023 characters):
APPLEBANANA10298
Enter 16-byte key:
APOISLKDJFHGMNXZ

Ciphertext (hex):
9c216d452796f8cf4b14c0d41568114f8ce4f51e85f881011eb98bb865564383
Decrypted Plaintext:
APPLEBANANA10298
请按任意键继续. . .

```

4. 建议使用由单词组成的密码短语（如 `servenoterampartoriginal`），不需要包含数字、特殊字符或更改大小写。以前的建议是根据密码短语来选择密码，如 `FS&7Yago`，它可以从短语“four score and seven years ago”中派生出来。

A) 假设 Alice 选择的密码由 4 个随机选择的单词组成，并假设 Alice 的词汇量为 2^{15} 。Trudy 必须测试多少个密码才能对 Alice 的密码进行穷尽搜索？（3 分）

答：Alice 的词汇量为 $2^{15} = 32768$ 个单词，密码由 4 个随机选择的单词组成（假设无重复或允许重复，但独立选择）。总可能的密码数量为 $32768^4 = (2^{15})^4 = 2^{60}$ 。计算得： $2^{60} = 1,152,921,504,606,846,976$ （约 1.15×10^{18} ）。Trudy 需要测试这么多密码才能完成穷尽搜索。

B) 假设 Bob 选择了一个由 8 个字符组成的密码，包括 5 个小写字母、1 个大写字母、1 个数字和 1 个特殊符号。一共有 10 个数字、26 个大写字母和 26 个小写字母，并假设有 32 个特殊字符。那么 Trudy 必须测试多少个密码才能对 Bob 的密码进行穷尽搜索？（3 分）

答：Bob 的密码为 8 个字符：正好 5 个小写字母、1 个大写字母、1 个数字、1 个特殊符号。首先，选择位置：从 8 个位置中选 1 个大写、1 个数字、1 个特殊符号的位置（剩余 5 个为小写），排列数为 $\frac{8!}{5!1!1!1!} = 336$ 。然后，每个位置的选择：

(1) 5 个小写： 26^5

(2) 1 个大写： 26

(3) 1 个数字： 10

(4) 1 个特殊： 32 总密码数量 = $336 \times 26^5 \times 26 \times 10 \times 32 = 33,214,624,235,520$ （约 3.32×10^{13} ）。Trudy 需要测试这么多密码才能完成穷尽搜索。

C) 根据这些数据，Alice 和 Bob 选择密码的方法哪个好？讨论每种选择密码的方法与其他方法相比可能具有的优势。（2 分）

答：根据数据，Alice 的选择方法更好。Alice 的密码空间约为 1.15×10^{18} （ 2^{60} ，约 60 比特熵），远大于 Bob 的 3.32×10^{13} （约 45 比特熵），大约是 Bob 的 34,711 倍，因此更抗穷尽暴力破解。

Alice 方法的优势：密码长度通常更长（4 个单词平均 20+ 字符）、易记（人类擅长记住短语）、无需强制复杂规则，减少用户写下或重用密码的风险，符合现代密码学推荐（如 NIST 指南强调长度而非复杂性）。

Bob 方法的优势：符合旧式密码政策（强制混合字符类型），可能在某些遗留系统中被要求；但缺点明显：短密码难记，导致用户倾向选择弱变体、重用或记录，实际安全性低，且易受针对性攻击。总体上，Alice 的方法在安全性和可用性上均优于 Bob 的传统复杂短密码。

D) 现在一般不建议定期更改密码，为什么？（2 分）

答：（1）强制定期更换会导致用户选择更弱、更可预测的密码（如在旧密码后加数字“Password1→Password2”），或重用旧密码变体，降低整体安全性；（2）用户可能因忘记新密码而频繁重置，增加支持负担和社会工程攻击风险；（3）此外，定期更换还会鼓励用户在多个站点重用密码。

5、实现摘要算法：SHA0 算法；

（1）最低要求：定长

（2）一般要求：不定长度

基本要求：给出算法结构描述，尤其是其迭代式结构，以及对明文长度的处理（加上段结束符后，对段落的划分）

评分细则：给出长度处理流程图及函数功能表述（3），画出算法流程图（3），编译正确（2），结果正确（2）

算法流程：

先将不足 512 位倍数的数据填充，先添加一个 1 位（字节表示为 0x80），再添加若干 0 位，直到填充后数据的位长度模 512 等于 448（即可用来存储原始数据长度），最后将原始消息的位长度（64 位小端序表示）附加到填充后的数据末尾。

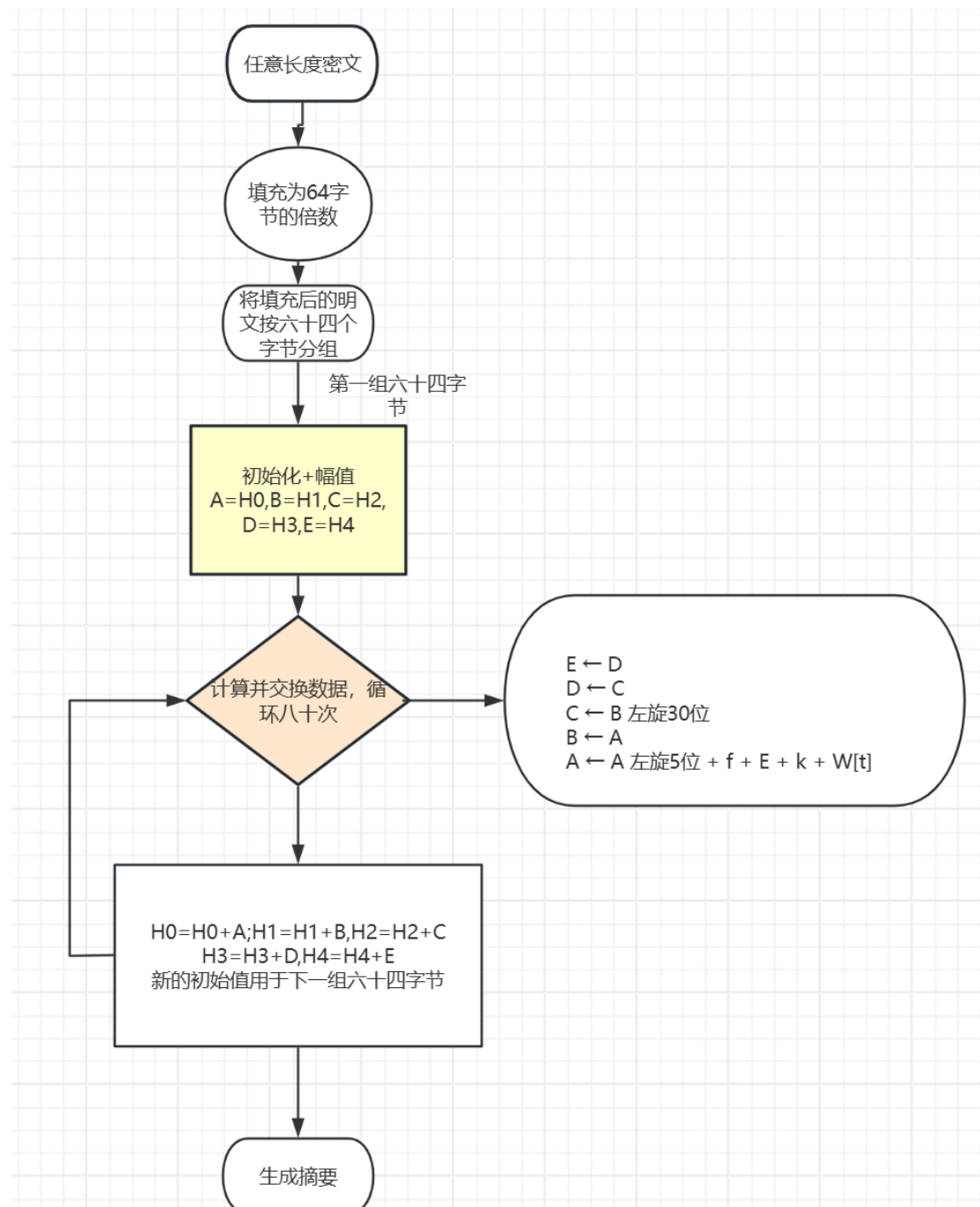
申请 5 个 32 位的链接变量（缓冲区寄存器），记为 H0、H1、H2、H3、H4，并使用 SHA-

0 标准指定的初始值进行初始化: $H_0 = 0x67452301$ $H_1 = 0xEFCDAB89$ $H_2 = 0x98BADCFE$ $H_3 = 0x10325476$ $H_4 = 0xC3D2E1F0$

最初填充后的数据被分为多个 512 位的块, 并被逐个独立处理。每处理一个块, 首先将该块按大端序拆分为 16 个 32 位字 $W[0..15]$, 然后通过异或运算扩展为 80 个字 $W[0..79]$ ($W[t] = W[t-3] \oplus W[t-8] \oplus W[t-14] \oplus W[t-16]$, 无旋转), 接着初始化工作变量 $A=H_0$ 、 $B=H_1$ 、 $C=H_2$ 、 $D=H_3$ 、 $E=H_4$, 随后执行 80 轮迭代运算, 每轮主要包括非线性布尔函数 $f(t, B, C, D)$ 、常量加法 $K[t]$ 、消息字 $W[t]$ 加法、 A 的 5 位循环左移以及 B 的 30 位循环左移等操作。

处理完所有 512 位块后, 将每轮计算得到的增量累加到五个缓冲区变量 $H_0 \sim H_4$ 上, 最终将更新后的 H_0 、 H_1 、 H_2 、 H_3 、 H_4 按大端序拼接在一起, 形成最后的 160 位 (40 个十六进制字符) 哈希值。

流程图如下



```

#include <iostream>
#include <string>
#include <vector>
#include <cstdint>
#include <iomanip>
#include <sstream>
using namespace std;

/**
 * SHA-0 哈希算法实现类
 * SHA-0 是 NIST 于 1993 年发布的原始 Secure Hash Algorithm
 * 输出 160 位（40 个十六进制字符）摘要
 * 注意：SHA-0 已存在已知碰撞漏洞，仅用于教学和历史研究
 */
class SHA0 {
private:
    // 五个 32 位状态寄存器（H0 ~ H4），初始值为 SHA-0 标准指定（大端序）
    uint32_t state[5] = {
        0x67452301,
        0xEFCDA89,
        0x98BADCFE,
        0x10325476,
        0xC3D2E1F0
    };

    // 辅助函数：32 位循环左移（防止移位量 >= 32 导致未定义行为）
    static uint32_t rotate_left(uint32_t x, uint32_t n) {
        n %= 32;
        return (x << n) | (x >> (32 - n));
    }

    /**
     * 消息填充（Padding）
     * 按照 Merkle-Damgård 结构要求，将任意长度消息填充为 512 位的倍数
     * 步骤：
     * 1. 追加一个 '1' 位（字节 0x80）
     * 2. 追加若干 0 字节，使填充后长度 mod 512 = 448 位（即 mod 64 字节 = 56 字节）
     * 3. 追加原始消息的位长度（64 位小端序，低位先）
     */
    void pad(const vector<uint8_t>& input, vector<uint8_t>& output) {
        output = input; // 复制原始消息
    }
};

```

```

        output.push_back(0x80); // 追加 '1' 位 (0x80 =
10000000)

    // 补 0 直到长度 mod 64 == 56 (剩余 8 字节用于长度)
    while ((output.size() % 64) != 56) {
        output.push_back(0x00);
    }

    // 计算原始消息的位长度 (字节数 * 8)
    uint64_t bit_len = static_cast<uint64_t>(input.size()) * 8;
    // 追加 64 位长度 (小端序, 8 个字节)
    for (int i = 0; i < 8; ++i) {
        output.push_back(static_cast<uint8_t>(bit_len >> (i * 8)));
    }
}

/**
 * 处理一个 512 位 (64 字节) 消息块
 * 这是 SHA-0 的核心压缩函数
 */
void process_block(const uint8_t* block) {
    uint32_t W[80]; // 消息调度数组, 前 16 个直接来自块, 后 64 个扩
展得到

    // 第一步: 将 64 字节块按大端序拆分为 16 个 32 位字 W[0..15]
    for (int i = 0; i < 16; ++i) {
        W[i] = (uint32_t(block[4 * i]) << 24) |
            (uint32_t(block[4 * i + 1]) << 16) |
            (uint32_t(block[4 * i + 2]) << 8) |
            uint32_t(block[4 * i + 3]);
    }

    // 第二步: 消息扩展 (SHA-0 关键特征: 无左旋 1 位)
    // 对于 t = 16 到 79: W[t] = W[t-3] ⊕ W[t-8] ⊕ W[t-14] ⊕
W[t-16]
    for (int i = 16; i < 80; ++i) {
        W[i] = W[i - 3] ^ W[i - 8] ^ W[i - 14] ^ W[i - 16];
    }

    // 初始化工作变量 A~E 为当前状态
    uint32_t A = state[0];
    uint32_t B = state[1];
    uint32_t C = state[2];

```

```

uint32_t D = state[3];
uint32_t E = state[4];

uint32_t temp, f, k; // 临时变量

// 第三步：80 轮主循环（分为 4 组，每组 20 轮）
for (int t = 0; t < 80; ++t) {
    // 选择非线性函数 f 和常量 k
    if (t < 20) {
        f = (B & C) | (~B & D); // f = (B AND C) OR
(NOT B AND D)
        k = 0x5A827999;
    }
    else if (t < 40) {
        f = B ^ C ^ D; // f = B XOR C XOR D
        k = 0x6ED9EBA1;
    }
    else if (t < 60) {
        f = (B & C) | (B & D) | (C & D); // f = (B AND C) OR
(B AND D) OR (C AND D)
        k = 0x8F1BBCDC;
    }
    else {
        f = B ^ C ^ D; // f = B XOR C XOR D
        k = 0xCA62C1D6;
    }

    // 核心更新公式
    temp = rotate_left(A, 5) + f + E + k + W[t]; // A 左旋 5
    位 + f + E + k + W[t]
    E = D; // E ← D
    D = C; // D ← C
    C = rotate_left(B, 30); // C ← B
    左旋 30 位
    B = A; // B ← A
    A = temp; // A ←
temp
}

// 第四步：将本块计算结果累加到全局状态
state[0] += A;
state[1] += B;
state[2] += C;
state[3] += D;

```

```

        state[4] += E;
    }

    // 重置状态为初始值（每次计算新哈希前调用）
    void reset() {
        state[0] = 0x67452301;
        state[1] = 0xEFCDAB89;
        state[2] = 0x98BADCFE;
        state[3] = 0x10325476;
        state[4] = 0xC3D2E1F0;
    }

public:
    /**
     * 计算输入字符串的 SHA-0 哈希值
     * @param input 任意长度的输入字符串
     * @return 40 个大写十六进制字符的摘要字符串
     */
    string hash(const string& input) {
        reset(); // 重置状态

        // 将字符串转为字节向量
        vector<uint8_t> bytes(input.begin(), input.end());

        // 填充消息
        vector<uint8_t> padded;
        pad(bytes, padded);

        // 逐块处理（每块 64 字节）
        for (size_t i = 0; i < padded.size(); i += 64) {
            process_block(&padded[i]);
        }

        // 格式化输出: 5 个 32 位字直接转为大写十六进制(大端序已保持)
        stringstream ss;
        ss << hex << uppercase << setfill('0');
        for (int i = 0; i < 5; ++i) {
            ss << setw(8) << state[i]; // 每个字占 8 位十六进制
        }
        return ss.str();
    }
};

// 主函数：测试用

```

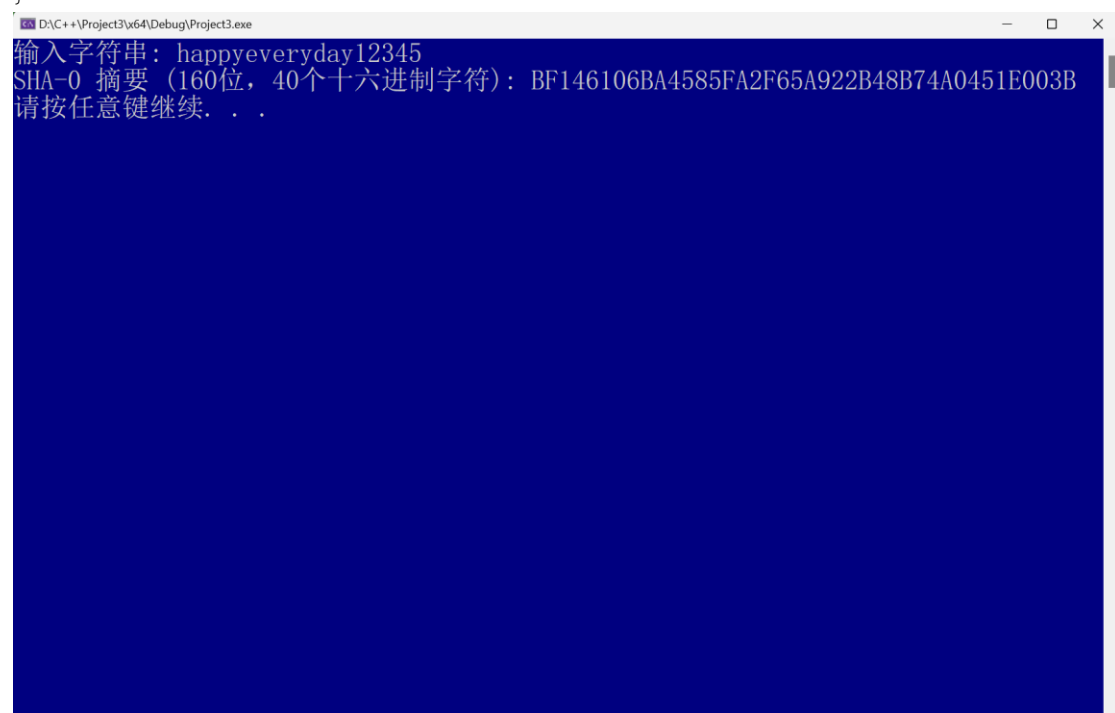
```

int main() {
    SHA0 sha0;
    string input;
    cout << "输入字符串: ";
    getline(cin, input); // 支持包含空格的输入

    string digest = sha0.hash(input);
    cout << "SHA-0 摘要 (160 位, 40 个十六进制字符): " << digest <<
endl;

    system("pause");
    return 0;
}

```



```

D:\C++\Project3\Debug\Project3.exe
输入字符串: happyeveryday12345
SHA-0 摘要 (160位, 40个十六进制字符): BF146106BA4585FA2F65A922B48B74A0451E003B
请按任意键继续. . .

```

6、请解释弱碰撞和强碰撞的概念，并举例比较弱碰撞和强碰撞的概率大小。山大王晓云院士是破译的什么密码算法。

弱碰撞：给定一个特定的输入 M ，找到另一个与 M 不同的输入 M' ，使得哈希函数的输出满足 $H(M) = H(M')$ ，且这个过程在计算上是不可行的。简单来说，就是针对已知消息找碰撞。

强碰撞：无需指定初始输入，任意找到两个不同的输入 M 和 M' ，使得 $H(M) = H(M')$ ，且这个过程在计算上是不可行的。也就是无差别地找任意两条消息的碰撞。

概率大小比较示例：

弱碰撞

给定输入“HAPPY”，它的哈希值为“1234567890QWER”。

我们可以根据已知消息尝试找到另一个字符串，比如“BIRTHDAY”，它的哈希值也是“1234567890QWER”。

强碰撞

找到两个完全无关的字符串，比如“COMPUTER”和“COUNTRY”，它们的哈希值相同。

王小云教授破译了 MD5、HAVAL-128、MD4 、RIPEMD、SHA-1 在内的五个国际通用哈希函数算法。

6、文件中存储的密码如何处理

A) 为什么对存储在文件中的密码进行散列处理是个好主意？

答：因为散列是一种单向函数：它可以将明文密码转换为固定长度的不可逆哈希值。即使攻击者窃取了存储的哈希值，也无法直接逆转得到原始密码，只能通过暴力破解（brute-force）或字典攻击尝试猜测。验证时，系统只需将用户输入的密码重新散列后与存储值比较即可。这比存储明文密码安全得多，明文一旦泄露，攻击者即可直接登录用户账户。

B) 为什么对存储在文件中的密码进行散列处理比对密码文件进行加密要好？

答：散列比加密（encryption）更适合密码存储，因为加密是双向的：它需要密钥来解密恢复明文。如果攻击者同时获得加密密码文件和解密密钥（密钥往往存储在单一系统或可访问处），就能轻松恢复所有明文密码，导致大规模泄露。散列则是单向的，无需存储任何可逆密钥，即使攻击者拿到散列值，也无法直接得到明文，只能逐个尝试破解（且使用慢散列算法如 PBKDF2 或 bcrypt 时成本极高）。加密适合需要恢复原数据的场景（如用户地址），但密码验证只需比较散列即可，无需恢复明文。因此，散列避免了密钥管理风险，提供更高安全性

C) 什么是“盐”？为什么在散列密码时要加盐？

答：“盐”（salt）是为每个密码随机生成的唯一字符串（通常至少 32 位随机字节），在散列前与密码拼接（如 `hash(salt + password)`），并与最终哈希值一起明文存储。盐本身不需保密。“加盐”的主要原因有以下几点：

（1）防止相同密码产生相同哈希：无盐时，多个用户用同一密码会生成相同哈希，攻击者可通过预计算的彩虹表（rainbow table）快速破解大量账户。

(2) **抵御预计算攻击**：盐使每个哈希独特，攻击者无法使用通用彩虹表或字典表，必须为每个用户单独计算，极大增加破解的时间和成本。

(3) **防御批量破解**：即使数据库泄露，攻击者需逐个针对每个盐进行暴力或字典攻击，无法一次性破解多个相同密码的用户。

8、论述你对信息安全的理解；分析信息安全正在以及将要对我们的生活带来的影响；预测未来信息安全的发展趋势。

一、对信息安全的理解

信息安全的核心目标，是在信息的存储、传输与使用的全生命周期中，确保其不被未经授权的访问、泄露、篡改或中断。从本质上讲，它是对信息的保密性、完整性、可用性、可控性与可靠性进行立体化、系统化的综合保障。信息安全并非单纯的技术命题，而是横跨个人、企业、社会乃至国家层面的复杂体系工程，它既需要密码学、网络安全、系统安全等技术手段，也离不开规范化的管理制度、完善的法律法规，更依赖公众的安全意识持续提升，从网络安全、数据安全到应用安全与物理安全协同构建起数字时代的“安全底座”。

在具体实现层面，信息安全至少包含以下关键维度：

第一，机密性。机密性是信息安全的基础，旨在确保敏感信息仅能供授权主体访问。在数字化社会中，国家安全文件、企业商业机密、个人隐私数据均以数字形态储存于网络系统，一旦泄露后果难以估量。因此需要利用加密技术、访问控制、身份鉴别等机制构建访问边界，从源头上降低信息泄露风险。

第二，完整性。完整性要求确保数据在存储、传输与处理过程中未被恶意篡改或意外损坏。常见的保护机制包括哈希校验、完整性验证、数字签名等手段，使任何对数据的非法修改能够被及时发现和追溯。

第三，可用性。可用性强调信息系统在需要时必须保持可访问与可操作状态。无论是硬件故障、网络攻击还是自然灾害，都可能导致系统瘫痪。因此，需要通过容灾备份、冗余设计、负载均衡与应急响应机制确保系统连续运行，为关键业务提供可靠支撑。

第四，可信度与不可抵赖性。可信度确保信息来源真实、内容可靠、流程可验证；不可抵赖性保证通信双方无法否认自身行为，对于电子商务、司法取证、数字交易等场景至关重要。数字签名、日志审计等技术在其中发挥核心作用。

此外，身份认证、访问控制、防火墙、入侵检测、漏洞扫描、安全审计等技术体系构成了现代信息安全的底座。而信息安全也并非静态状态，而是随着威胁演变不断更新的持续过程，涉及组织文化建设、制度设计、风险管理实践与安全意识教育等多维度协同。

二、信息安全对个人、企业与社会生活的影响

信息安全已经深度融入并持续影响现代社会的各个层面，其影响至少体现在以下三个维度：

（一）对个人生活的影响

在高度数字化的环境中，个人隐私泄露成为社会广泛关注的问题，身份信息、金融账户、地理位置、消费行为均可能被非法利用，造成财产损失甚至人身风险。同时，网络诈骗、账户盗用、虚假信息传播对个人安全构成持续威胁。信息安全水平的提升，意味着更可靠的隐私保护、更安全的支付体验和更可信的社交环境，有助于增强用户对数字生活的信任与参与。

度。

（二）对企业运营与经济活动的影响

企业面临的网络攻击手段正快速演进，勒索软件、供应链攻击、内部威胁等频发，安全事件往往造成巨额经济损失与声誉损害。信息安全不仅成为企业成本，更成为与研发、运营同等重要的战略能力。与此同时，数据安全法规与合规要求不断加强，例如 GDPR、网络安全法等，使企业在跨国经营、数据共享、客户管理等方面必须具备更高的合规能力。

（三）对国家治理与社会稳定的影响

网络空间已成为国家安全的关键领域。关键基础设施——如能源、电力、金融、交通和医疗系统——均依赖稳定的网络环境，一旦遭受攻击，可能引发社会性风险。信息安全同时承担着支撑数字政府、数字经济、智慧城市等现代治理体系的职责，确保社会运行的可靠性与秩序性。

总体而言，信息安全的影响已经超越个体与组织，成为国家竞争力的重要组成部分。

三、未来信息安全的发展趋势

面向未来，随着人工智能、物联网、量子计算等技术快速发展，信息安全将呈现多维度、智能化的变革趋势，包括：

1. 人工智能驱动的安全体系成为主流

人工智能将为威胁检测、自动化响应、攻防测试、安全策略优化提供更能力，使安全运营从人工指定规则向智能化、预测性转变。然而，AI 模型本身也可能成为攻击目标，模型投毒、对抗样本等新型威胁需要同步解决。

2. 数据隐私保护与隐私计算技术加速应用

全球范围内的数据隐私法规不断收紧，隐私保护需求前所未有地增长。联邦学习、同态加密、安全多方计算等技术将成为数据共享与合规处理的关键支撑，使“可用不可见”的数据模式得以普及。

3. 信息安全人才缺口持续扩大

世界范围的信息安全岗位长期供不应求，威胁技术更新迅速，人才培养周期长。因此，各国将加大在安全教育、产学研合作和职业培训方面的投入，信息安全教育也将向更低年龄段延伸。

5. 量子安全体系逐步成型

量子计算对现有公钥密码构成潜在威胁，推动后量子密码（PQC）和量子密钥分发（QKD）的研究与应用加快落地，未来在金融、国防、通信等关键领域率先使用